Visibility-Aware Task and Motion Planning with Task-Centric Perception

Anonymous Authors

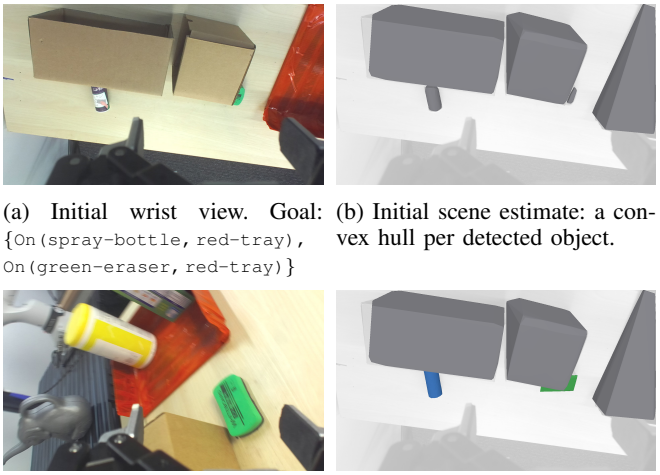
***Abstract*— Long-horizon manipulation under uncertainty remains a challenging problem in robotics. We present `VA-TAMP`, an algorithm for Visibility-Aware Task and Motion Planning. `VA-TAMP` extends existing deterministic task and motion planning solvers to partially observable object rearrangement tasks, while still being able to take advantage of the efficiency and compositional generalization over long horizons of the base solver. This extension interleaves sensing actions into a plan skeleton that assumed full observability, and uses a novel persistent object-based memory to fuse observations into object mesh hypotheses and an observed occupancy map. We evaluate `VA-TAMP` on real-world tabletop rearrangement tasks which feature heavy occlusion and partial visibility and demonstrate that it outperforms baselines that do not perform active perception or have persistent memory.**

I. INTRODUCTION

A longstanding challenge in robotic manipulation is completing long-horizon tasks in heavily occluded environments with unfamiliar objects. When object models and poses are known *a priori*, Task and Motion Planning (TAMP) solvers [1], [2] can efficiently compose manipulation skills to achieve a goal. Otherwise, object models can be estimated from an observation and passed to a TAMP solver [3], [4]. These approaches assume that the initial observation models the scene accurately enough to find a goal-achieving plan.

Consider a tabletop rearrangement task, such as the one illustrated in Figure 1, where the robot has to place the cleaning supplies on the red tray. In the initial observation, the full extent of the tray (which already has other cleaning supplies on it) is not visible to the robot, and the cleaning supplies that the robot has to manipulate are partially occluded by the cardboard boxes. This setting is partially observable; the robot does not know the full shape of any of the objects in the scene, and thus it must make a plan from raw observations which are noisy and incomplete.

The naive approach to this problem, re-estimating and replanning with every new observation, suffers from two main shortcomings. First, any single observation is not sufficient to accurately model the scene. Second, TAMP solvers output strictly goal-oriented plans, meaning that they cannot direct the perception toward the task-relevant parts of the scene or remove occluding objects. In Figure 1, no single view shows the far edge of the tray, and a plan that only reaches for the eraser never looks behind the boxes that occlude it. On the other end of the spectrum, belief space planners explicitly model uncertainty in the environment state and search for plans by reasoning over full observation distributions obtained by each action [5], [6], [7], [8]. However, modeling geometric uncertainty requires reasoning over extremely high-dimensional distributions which quickly becomes intractable.



(a) Initial wrist view. Goal: {`On(spray-bottle, red-tray)`, `On(green-eraser, red-tray)`}

(b) Initial scene estimate: a convex hull per detected object.

(c) A planned wrist view reveals the full extent of the eraser.

(d) Task-relevant objects refined into completed shapes.

Fig. 1: Visibility-Aware Task and Motion Planning

Our approach exploits the compositional generalization and efficiency of TAMP solvers while tracking object model accuracy and unobserved regions of space. `VA-TAMP` augments off-the-shelf TAMP solvers with action constraints requiring the robot to verify its object models before interacting with the objects. These constraints induce information gathering actions that provide observations for refining the models.

In order to build and refine our scene representation as we observe more of the scene and objects move, we define an *Object-Based Memory* module (OBM). The OBM collects a history of observations and maintains two complementary representations of the scene. The first representation is an occupancy grid, which accumulates depth data and tracks the regions of space that have not yet been observed. The second representation is a set of objects that are believed to exist in the scene after being detected in individual observations. The OBM associates these detections on a per-object basis and leverages a generative image-to-3D reconstruction model [9], [10] to hypothesize a complete 3D mesh geometry from partial RGBD observations. The OBM is resource-rational and task-centric: it focuses its computation time on objects that the planner has decided are relevant to the task. In the example, the eraser and tray are refined into complete meshes (Figure 1d) while the cardboard boxes are left as the coarse hulls from the first frame (Figure 1b).

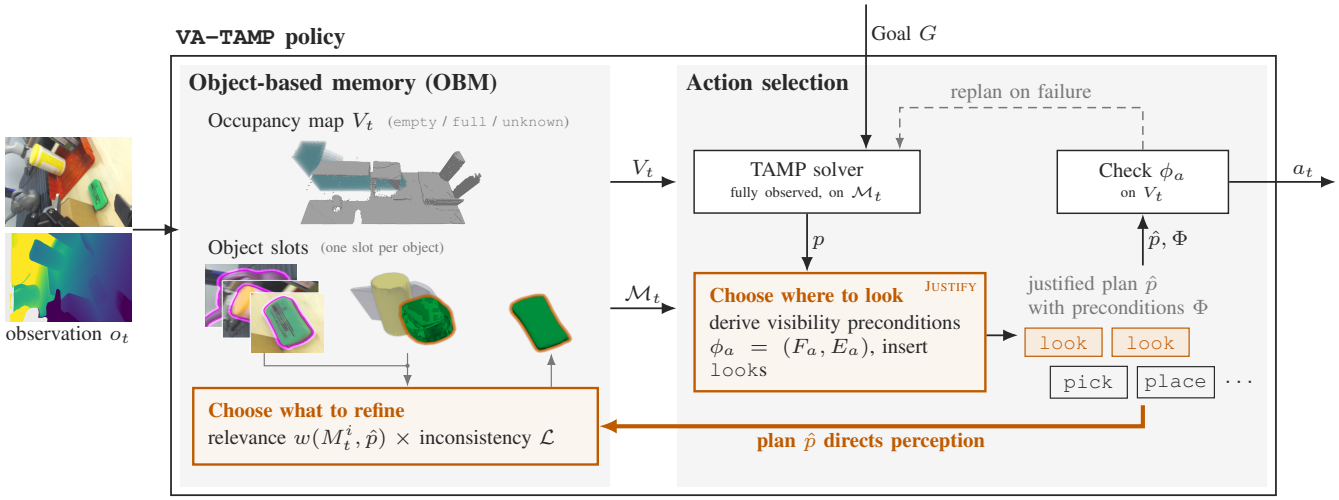


Fig. 2: Overview of VA-TAMP. RGB-D observations update an object-based memory (OBM), which maintains observed occupancy V_t and object mesh hypotheses $\mathcal{M}_t$. Given a goal G, a fully observed TAMP solver computes a plan p over $\mathcal{M}_t$. JUSTIFY uses $\mathcal{M}_t$ and V_t to derive a set of visibility preconditions Φ and insert look actions to produce a justified plan $\hat{p}$. These preconditions are checked against V_t to select the next action a_t or trigger replanning. The plan also guides OBM refinement, prioritizing mesh hypotheses that are relevant to the plan and inconsistent with accumulated observations.

Given a set of object meshes, we can leverage existing classical TAMP solvers to generate a plan skeleton: an optimistic plan that assumes the current object meshes are accurate and is entirely goal-oriented. To guard against this optimism, we require that every plan be *justified*. This means that any time the plan requires the robot to move through space, it must have observed that the space is collision free. Any time the plan requires the robot to grasp an object at a given pose, it must have observed that the object surface exists in between the fingers at that pose. To justify a plan, the planner has to find motions that move the camera to viewpoints where the desired regions are visible. The initial hull of the green eraser (Figure 1b) does not justify a grasp; the planner inserts the wrist view in Figure 1c, which reveals the eraser's full extent and lets the grasp proceed.

VA-TAMP extends fully observed TAMP to partially observed domains by coupling object-based memory with scene-model verification. The memory supplies object meshes and an occupancy grid, and the planner constructs a goal-reaching plan that also validates the meshes. As observations arrive, the remaining plan guides refinement of task-relevant objects. Figure 2 illustrates this coupling. Our contributions are as follows.

VA-TAMP: An algorithm for Visibility-Aware Task and Motion Planning. VA-TAMP augments classical TAMP solvers with a persistent memory system and task-driven information gathering behavior. We show that this method outperforms systems which don't perform active perception or do not associate object detections across observations.

A task-centric, resource-rational OBM. Accurately modeling a scene is a computationally expensive task, with state-of-the-art completion models taking 4-5 seconds per object in a single frame. Rather than completing every object in every frame, the OBM prioritizes reconstruction on objects that are task-relevant and do not yet have a good model.

Efficient information gathering in partially observable problems by leveraging a fully observed TAMP solver. We use the result of an existing TAMP solver to define which regions of space the robot interacts with, either as motion in assumed free space or contact with assumed object surfaces. We make the observation of these regions of space a precondition to acting, forcing the robot to collect more observations and update its models as needed.

II. RELATED WORK

A. Visibility-Aware Planning

Goretkin et al. [11] formulate visibility-aware motion planning by requiring a robot to observe a region as empty before moving through it. Muguira-Iturralde et al. [12] extend this formulation to allow the robot to move occluders on the trajectory. Ashur and Sintov [13] instead treat visibility as an objective and seek configurations with a clear line of sight to a target. Next-best-view planners [14], [15] select views to collect information efficiently, but are generally task-agnostic. Our method uses visibility constraints to verify both free space along planned motions and object surfaces at planned grasp contacts. The resulting observations also guide refinement of the object geometry.

B. Task and Motion Planning

Task and Motion Planning (TAMP) is a well-studied problem formulation, and here we focus on TAMP approaches that operate without access to the ground-truth state. Curtis et al. [3] and Shen et al. [4] estimate the scene from an initial observation and treat this estimate as ground truth for a deterministic TAMP solver. VA-TAMP builds on the same

class of solvers, but verifies and updates the estimate as the plan executes.

Another class of TAMP approaches plan in belief space using determinized observation models [16], [17] or by weighting observation outcomes by their likelihood [18]. These approaches replan when new observations invalidate the current plan. Curtis et al. [19] avoid determinization by learning a belief-space MDP from simulated operator rollouts and solving that MDP. Our OBM instead keeps a single mesh hypothesis per object at a time and evaluates how well it explains the accumulated observations, without explicitly maintaining a distribution over possible geometries.

Recent work has also used Vision-Language Models (VLMs) to handle uncertainty in TAMP problems. Zhao et al. [20] use a VLM to classify predicates as known or unknown and plan observation actions to resolve the unknown ones. Tanaka et al. [21] and Oloo et al. [22] use VLMs to propose observation actions and decide whether the robot has enough information to act. These methods use VLMs to reason about symbolic uncertainty and select observations. VA-TAMP instead addresses continuous geometric uncertainty by deriving observation targets from planned motions and contacts.

C. Scene Representation

Mitash et al. [23] choose each object's representation according to its role in the plan. Objects to be grasped are represented by minimal alpha shapes, which restrict grasps to observed surfaces. Objects to be avoided are represented by conservative hulls that include their unobserved shadows. Our OBM instead updates object meshes throughout the episode, using the observations to evaluate each mesh and the plan to prioritize refinement.

To fuse observations over time, Gu et al. [24] build a persistent 3D scene graph by associating objects using geometric overlap and visual feature similarity. Du et al. [25] learn an object-based memory that updates each slot's shape estimate as new observations arrive, including in scenes where objects move.

Task-agnostic reconstruction methods use generative image-to-3D models to estimate complete object meshes and poses from sparse views [26], [27], [28]. Zadaianchuk et al. [10] jointly estimate object pose, scale, and geometry. Our OBM instead updates a persistent scene representation as observations arrive and objects move, while allocating reconstruction effort according to the current manipulation plan.

III. Partially Observable Object Rearrangement

A. Problem Statement

We formulate long-horizon manipulation under partial observability as a Partially Observable Object Rearrangement (POOR) problem, an instance of a stochastic shortest-path problem on a belief space MDP whose underlying state is defined in terms of posed meshes.

The robot manipulator has a set of fully observed joint angles q and is embedded in an environment with an unknown number N^o of objects. Each object in the object set $\mathcal{M}$ is a posed mesh $M^i = (m^i, X^i) \in \mathcal{M}$, where m^i is a tuple of surface vertices and faces, and X^i is the pose of the mesh centroid relative to a known world frame. Neither the object geometries nor their poses are provided a priori, and both must be inferred from observations. The state space $\mathcal{S}$ of a POOR problem is the space of possible sets of geometries and robot configurations.

At every step, the robot takes a temporally extended parameterized action $a \in \mathcal{A}$ (e.g., $\texttt{Pick}(X^{\texttt{hand}})$, $\texttt{Place}(X^{\texttt{hand}})$, $\texttt{Look}(X^{\texttt{hand}})$), which returns an observation sequence $o_{1:T} \in \mathcal{O}$. Each observation $o_t = (q_t, \mathcal{I}_t)$ contains the robot's joint angles q_t and an RGBD image $\mathcal{I}_t$ captured at time t. We assume access to a transition function $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ which describes how actions change the state.

The robot is given an initial observation o_0 and a goal G, which is a function on the history of observations that is true when a desired property holds in the world. For example, given a text label of an object l_0 and desired support surface l_s, the robot may try to achieve the goal $\exists x, y \; s.t. \; \texttt{Is}(x, l_0) \wedge \texttt{Is}(y, l_s) \wedge \texttt{On}(x, y)$. The solution to a POOR problem is a policy $\pi : \mathcal{O} \to \mathcal{A}$ which maps observations to actions, is executed starting from the initial observation, and is evaluated by computing the cost of the actions taken to move from the initial state to a goal-satisfying state.

B. Overall Approach

Computing a full policy mapping from any observation to the correct action completely offline is impractical due to the complexity of the POOR problem. Instead, VA-TAMP takes a model-predictive control style approach to constructing a policy. We combine a state estimator which computes models of the scene conditioned on observations with an action-selection module which selects an action conditioned on the goal and current scene models. On execution of the action, the policy receives a new observation, which it uses to update its model and continue execution or replan.

This approach to constructing the policy is illustrated in Algorithm 1. We initialize the *Object-Based Memory* from the initial observation (line 1) and use this to estimate a hypothesized set of meshes in the world and a voxel occupancy grid (line 3). The action selection (lines 4-5) starts by solving a fully observed TAMP problem assuming $\mathcal{M}$ is the true state to get a plan backbone p. The JUSTIFY procedure derives observation-conditioned preconditions for manipulation actions and then interleaves sensing actions into the plan to verify the preconditions to get the justified plan $\hat{p}$. If the preconditions hold in the true scene, we execute the actions in the plan and receive new observations, which are used to update the OBM. If the preconditions do not hold, the OBM is queried for a fresh scene model to plan against.

In Section IV, we describe the *resource-rational* OBM, which trades perception computation for utility in downstream planning. In Section V we define the JUSTIFY

Algorithm 1 VA-TAMP

Require: goal G, initial observation o_0, robot pose q_0

$\mathcal{B} \leftarrow$ INIT-OBM(o_0)
while $\neg G$ **do**
  $\mathcal{M}, V \leftarrow$ ESTIMATE-SCENE$(\mathcal{B})$
  $p \leftarrow$ SOLVE-TAMP$(q_0, \mathcal{M}, G)$
  $\hat{p}, \Phi \leftarrow$ JUSTIFY$(p, \mathcal{M}, V)$
  for $a, \phi_a \in \hat{p}, \Phi$ **do**
    if $\phi_a(V_t, \mathcal{M}_t)$ **then**
      $o_{1:t}, q_t \leftarrow$ EXECUTE(a)
      UPDATE$(\mathcal{B}, o_{1:t}, \hat{p})$
    else
      break ▷ replan

procedure used to derive action preconditions and sensing actions.

IV. Task-Centric Object-Based Memory

A. Problem Statement

The OBM is responsible for computing up-to-date representations of the scene to be used by a downstream planner. Formally, the OBM takes as input an observation history $o_{1:t}$ and the robot's current plan $\hat{p}$. It returns a collection of posed surface meshes $\mathcal{M}_t$ and an occupancy map V_t. Generally, occupancy maps associate every voxel with a probability of that voxel being occupied [29], [30]. Our occupancy map is a discretized special case of this and takes the form

$$V^r(x, y, z | o_{1:t}) \in \{\texttt{empty}, \texttt{full}, \texttt{unknown}\}.$$

Here (x, y, z) is any point in the bounded workspace, expressed in the world frame, and V^r returns the label of the voxel of side length r containing it.

Estimating $\mathcal{M}_t$ requires hypotheses about unobserved object surfaces. Thus, we assume access to a generative shape model $\mathcal{G}$ that maps RGBD observations of an object to a posed mesh hypothesis for the full object. Querying $\mathcal{G}$ is slow relative to the observation frequency, so the OBM makes at most one query per *perception step*, running in parallel with robot execution. Estimating $\mathcal{M}_t$ is therefore a sequential decision problem: at each step, the OBM chooses which object to refine and whether the returned mesh should replace its current hypothesis.

The OBM is evaluated on the quality of its mesh representation $\mathcal{M}_t$. Let M_t^i denote the posed mesh estimate of object i at perception step t, and let M^i denote its true posed mesh. Given the plan, the OBM's goal is to minimize

$$J(\mathcal{M}_{0:T}) = \sum_{t=0}^{T} \sum_{i=1}^{N^o} w(M_t^i, \hat{p}_t)\, e(M_t^i, M^i),$$
$$w(M_t^i, \hat{p}_t) = \exp\Big(-\beta \min_{x \in M_t^i,\ y \in \tau_{ee}(\hat{p}_t)} \|x - y\| \Big). \quad (1)$$

Here e measures the discrepancy between the estimated mesh and the true object, and $\tau_{ee}(\hat{p}_t)$ is the end-effector path swept by the current plan. Since the true meshes are unavailable at runtime, the OBM minimizes a surrogate, the cost of each mesh under its observations, defined in the following section.

B. Memory Representation and Updates

Our OBM maintains a set of *object slots*, each representing a hypothesized object through its observation history and current 3D geometry estimate. An object detection is $d_t^i = (o_t, \ell_t)$, where o_t is an observation containing the object and ℓ_t is a binary mask of its image pixels. We define a slot as

$$s_t^i = (M_t^i, \mathcal{D}_t^i, \mathcal{P}_t^i).$$

where $\mathcal{D}_t^i$ is the set of detections attributed to object i, and $\mathcal{P}_t^i$ contains the accumulated world-frame points obtained by back-projecting the depth pixels assigned to object i in these detections.

The object slots are updated based on each observation, and these slots are then used to update the mesh-based representation and occupancy-based representation. From every observation, a set of detections is computed. These detections are then either associated with an existing object slot, or initialize a new object slot. The OBM updates its mesh-based representation by choosing a slot to run the completion model on and updates its occupancy-based representation by aggregating the depth images and updating regions of space based on object rearrangement.

Acquiring and Associating Detections. For each observation o_t, we query a VLM for bounding boxes of objects on the table, then use a segmentation model to refine each box into an object mask. The resulting detections d_t^i are matched to existing slots or treated as novel objects.

To assign detections to the appropriate object slot, we follow a similar strategy to ConceptGraphs [24]. A new detection $d = (o, \ell)$ yields a set of world-frame points $\mathcal{P}(d)$, obtained by back-projecting the depth pixels under the mask ℓ. It also yields a visual feature $f(d)$, the mean of the dense DINO features [31] over the same pixels. The corresponding quantities for a slot s_t^i are its accumulated points $\mathcal{P}_t^i$ and the mean feature over its detections, $f(s_t^i) = \frac{1}{|\mathcal{D}_t^i|} \sum_{d' \in \mathcal{D}_t^i} f(d')$. We score a detection against a slot as

$$\text{score}(d, s_t^i) = \tfrac{1}{2}\big(g(d, s_t^i) + \cos(f(d), f(s_t^i)) \big),$$
$$g(d, s_t^i) = \max\big(c(\mathcal{P}(d) \to \mathcal{P}_t^i),\ c(\mathcal{P}_t^i \to \mathcal{P}(d)) \big),$$

where $c(P \to Q)$ is the fraction of points in P within δ of some point in Q. A slot is a candidate only if $g \geq \tau_g$ and $\cos(f(d), f(s_t^i)) \geq \tau_f$ for a specified geometric similarity threshold τ_g and visual similarity threshold τ_f. Within each observation, detections are processed in descending order of their best candidate score. Each detection joins the highest-scoring candidate slot that has not already been assigned, appending d to $\mathcal{D}_t^i$ and $\mathcal{P}(d)$ to $\mathcal{P}_t^i$, or seeds a new slot if there are none.

A detection d seeds a new slot by instantiating $s_t^i = (M_t^i, \{d\}, \mathcal{P}(d))$, with M_t^i denoting the convex hull of $\mathcal{P}(d)$.

Updating the Mesh Estimate. At every perception step, the OBM chooses which object model to update. Because the true geometries in the scene are unknown, we define a

surrogate cost which measures how inconsistent a candidate posed mesh M' is with the object detections $\mathcal{D}_t^i$ and points $\mathcal{P}_t^i$. The cost $\mathcal{L}$ is a negative log-likelihood given by

$$\begin{aligned}\mathcal{L}(M', \mathcal{P}, \mathcal{D}) = \lambda_{DDM}\, \mathcal{L}_{DDM}(M', \mathcal{P}) \\ + \lambda_{empty}\, \mathcal{L}_{empty}(M', \mathcal{D}) \\ + \lambda_{recall}\, \mathcal{L}_{recall}(M', \mathcal{D}),\end{aligned}$$

where $\mathcal{L}_{DDM}$ is the directional distance metric proposed by [32] and is a correspondence-free approach to measuring the alignment between observed points and the mesh. $\mathcal{L}_{empty}$ and $\mathcal{L}_{recall}$ are negative log-likelihoods comparing the mesh rendered at the proposed scale, rotation, and translation to the observations, derived from the Bayes3D perception system [33].

The OBM then greedily picks the slot whose current estimate M_t^i is the most inconsistent with its evidence, weighted by task relevance,

$$i^* = \arg\max_i \; w(M_t^i, \hat{p}_t)\, \mathcal{L}(M_t^i, \mathcal{P}_t^i, \mathcal{D}_t^i),$$

where w is the task-relevance weight defined in Equation 1.

In practice, the majority of current image-to-3D reconstruction models can only be conditioned on a fixed number of views, and not an arbitrarily long list of detections. Thus, after selecting i^*, the OBM selects the detection $d \in \mathcal{D}_t^{i^*}$ that reveals the largest fraction of object i^* and uses this as the input to $\mathcal{G}$. We then evaluate $\mathcal{L}_{\text{cand}} = \mathcal{L}(\mathcal{G}(d), \mathcal{P}_t^{i^*}, \mathcal{D}_t^{i^*})$ as the cost of the model output and $\mathcal{L}_{\text{curr}} = \mathcal{L}(M_t^{i^*}, \mathcal{P}_t^{i^*}, \mathcal{D}_t^{i^*})$ as the cost of the current slot estimate. If $\mathcal{L}_{\text{cand}} < \mathcal{L}_{\text{curr}}$, then the slot mesh $M_t^{i^*}$ is replaced with $\mathcal{G}(d)$.

Updating the Occupancy Grid. The voxel grid V is instantiated from the first observation o_0 and updated in real time as new observations arrive. We represent the voxel grid in memory as an octree [34] which stores an estimate of the log-odds of each cell being occupied and supports efficient Bayesian updates. A cell is marked `unknown` if it is not observed in any depth image, `empty` if the probability of the cell being occupied is less than 0.5, and `full` otherwise.

Picking up and placing an object requires an additional update to V because grasped objects occlude much of the visible area of a wrist mounted camera. When the robot picks up object i, the cells containing the corresponding slot's points $\mathcal{P}_t^i$ at the object's initial pose are marked as `unknown` in V. The cells are marked `unknown` and not `empty` because noise in the depth measurements and segmentation models means that $\mathcal{P}_t^i$ may contain points on the surfaces of objects that neighbor or support object i, so marking the cells `unknown` more accurately captures the uncertainty about the cell state after grasping.

V. Plan Justification

A. Problem Statement

The input to plan justification is a plan $p = (a_1, \ldots, a_n)$ from a TAMP solver that treated the OBM's mesh hypotheses $\mathcal{M}_t$ as ground truth. Each action a_u has a target robot pose X_{a_u}, a joint-space trajectory τ reaching that pose, a target object o^u, and a constraint set $\texttt{con}(a_u, \mathcal{M}_t)$. These constraints must all hold for the action to be valid; for example, X_{a_u} must not collide with any object in $\mathcal{M}_t$.

Algorithm 2 JUSTIFY

Require: unjustified plan p, mesh set $\mathcal{M}_t$, occupancy map V_t

1: $\hat{p} \leftarrow ()$; $\Phi \leftarrow \emptyset$
2: **for** $(\texttt{Pick}(o^u, X),\ \texttt{Place}(o^u, X')) \in p$ **do**
3: $\quad \phi_{\text{pick}} \leftarrow \textsc{DerivePre}(\texttt{Pick}(o^u, X), \mathcal{M}_t)$
4: $\quad \phi_{\text{place}} \leftarrow \textsc{DerivePre}(\texttt{Place}(o^u, X'), \mathcal{M}_t)$
5: $\quad \hat{p} \leftarrow \hat{p} \,\|\, \big(\texttt{Look}(\phi_{\text{pick}}),\ \texttt{Look}(\phi_{\text{place}})\big)$
6: $\quad \hat{p} \leftarrow \hat{p} \,\|\, \big(\texttt{Pick}(X),\ \texttt{Place}(X')\big)$
7: $\quad \Phi \leftarrow \Phi \cup \{\phi_{\text{pick}}, \phi_{\text{place}}\}$
8: **return** $\hat{p}, \Phi$

The output is a *justified* plan $\hat{p}$, which interleaves `Look` actions with the original manipulation actions in p, and a set of *visibility preconditions* $\Phi = \{\phi_a\}_a$. These preconditions are derived from p and take the form

$$\phi_a(V_t, \texttt{con}(a, \mathcal{M}_t)) \in \{\text{true}, \text{false}\} \tag{2}$$

and are tested before the action a executes. The precondition ϕ_a holds when the constraints on the hypothesized meshes $\mathcal{M}_t$ are supported by the observed occupancy grid.

A justified plan should be *valid*: before each manipulation action a_u, the preceding `Look` actions should have labeled every voxel its precondition depends on. It should also be *efficient*, avoiding observations that increase the original plan's cost without validating or helping refine a task-relevant object.

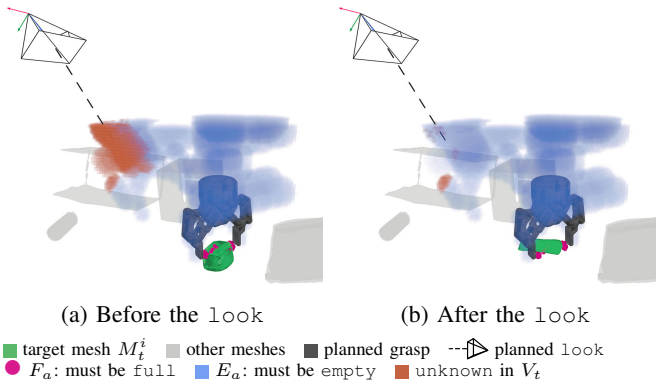
B. Constructing Justified Plans

The Justified Plan. Pseudocode for JUSTIFY is given in Algorithm 2. The plan p returned by the TAMP solver in Algorithm 1 is *unjustified*: it is valid against $\mathcal{M}_t$, but V_t may not contain observations that support every action in p. JUSTIFY assumes that the plan consists of a sequence of `Pick`, `Place` pairs. It starts by deriving the visibility preconditions for each of these actions (lines 3–4), then inserts `look` actions into the original plan (lines 5–6), and returns the *justified* plan $\hat{p}$ together with the set of preconditions Φ that Algorithm 1 checks before executing each manipulation action.

Looks are inserted by a fixed rule. For each `pick` and `place` pair in p, a `look` for the pick's precondition and another for the place's precondition are both inserted immediately before the `pick`. The place's look is not deferred until after the pick because, once the robot grasps something, the hand is no longer free and the wrist camera cannot collect new observations. Algorithm 1 then executes $\hat{p}$ in order, checking each precondition in Φ against the current V_t before its action runs and returning to the OBM for a fresh scene estimate when a check fails.

Deriving Visibility Preconditions. Every visibility precondition ϕ_a of Equation 2 is a tuple of point sets

$$\phi_a = \big(F_a, E_a\big), \qquad F_a, E_a \subset \mathbb{R}^3,$$



(a) Before the look (b) After the look

Fig. 3: A precondition for pick(eraser, g, q). (a) The precondition computed from an estimated $\mathcal{M}_t$. The finger contacts F_a (pink) must read full, and the swept volume of the gripper along the grasp trajectory E_a must read empty. The blue cells are known to be empty, the orange cells are unknown. (b) After the look, the required cells are observed and ϕ_a holds.

where every point in F_a must be labeled full and every point in E_a must be labeled empty. Figure 3 shows the precondition of a pick action derived while manipulating the green eraser in the motivating example of Section I, before and after the look that satisfies it.

DERIVEPRE computes ϕ_a from the constraint set on the meshes. The solver represents a Pick as $\texttt{pick}(o^u, X^{\text{grasp}}, \tau)$, with an object u and a grasp pose X relative to o^u, and a Place as $\texttt{place}(o^u, X^{\text{place}}, \tau)$, with a placement pose X^{place} in the world frame.

- $\texttt{pick}(o^u, X^{\text{grasp}}, \tau)$: the antipodal constraint on X^{grasp} [35] depends on the surface of o^u at the two finger contacts, so F_a contains the points on o^u where the finger pads close. The collision-free constraint is on the volume swept by the arm along τ while approaching the grasp, so E_a contains samples of this volume.
- $\texttt{place}(o^u, X^{\text{place}}, \tau)$: the support constraint on X^{place} starts by defining the "footprint" of o^u as the projection of the volume o^u onto a plane at the lowest point of o^u. F_a contains a column of points below each cell of the footprint, satisfied when any point in the column is labeled full, which would imply the footprint is supported. The collision-free constraint depends on the footprint at X^{place}, the volume swept by the held object as the robot follows τ, and the volume swept by the arm, so E_a contains samples of all three.

The visibility precondition ϕ_a holds when a fixed fraction of the constrained points have been observed and have the required label. Preconditions are derived once from the plan with respect to $\mathcal{M}_t$ and re-derived whenever $\mathcal{M}_t$ changes, since a new mesh hypothesis updates the contacts, footprint and swept volumes that define them.

The look Operator. Satisfying a visibility precondition requires new observations. The Look action is parameterized as $\texttt{look}(\phi_a, X)$, taking a precondition and a sequence of camera poses $X = (X_1, \ldots, X_m)$. A single pose is insufficient when the points of a precondition do not fit in one field of view, or are occluded from one direction and visible from another. Its precondition is that the hand is empty so that the wrist-mounted camera is unobstructed, its effect is that the points of ϕ_a have been labeled in V_t, and its constraint is that each pose in X is reachable and collision-free against $\mathcal{M}_t$.

The sampler for X is a greedy set cover over the subset of $F_a \cup E_a$ that is currently labeled unknown in V_t. We assume a camera model that predicts, for a candidate pose, which points fall in the field of view and are unoccluded by the meshes in $\mathcal{M}_t$. Each round selects an unknown point and samples candidate camera poses on the surface of a sphere around that point, oriented to look at the point. Samples are discarded if they are kinematically infeasible, are predicted to be occluded by other objects in the scene, or move the robot through points not labeled empty in V_t. Among the surviving samples, we keep the one whose field of view contains the most points in the precondition that are still unknown. Sampling continues until the chosen poses are expected to cover a fixed fraction of the precondition's required points or the view budget is exhausted. If no cover can be found, the precondition is marked unobservable; its precondition then fails at execution time and Algorithm 1 replans. To execute the look skill, a collision-free trajectory is planned that sequentially visits each viewpoint.

Because the underlying TAMP solver assumes full observability, it will not output actions that improve the visibility of the scene, although modifying the TAMP solver to move visual occluders is feasible. Therefore the Look actions assume that there is some viewpoint from which the region ϕ_a can be viewed, rather than planning actions to increase the visibility. In order to construct a solvable problem for the fully observed TAMP planner, we assume that the initial observation contains every object referenced in the goal statement. We also assume that while Look actions return noisy observations, they are not so noisy that collecting more observations worsens the quality of $\mathcal{M}_t$, and V_t. That is, collecting more observations should generally yield a better model of the world, not a worse one.

VI. EXPERIMENTS

Partially observable object rearrangement is an ubiquitious but surprisingly under-studied problem setting and, as such, there are no directly comparable existing methods. In this section, we explore the effectiveness of VA-TAMP in three scenarios of increasing complexity that were specifically designed to stress different aspects of the problem. We are interested in understanding when the complexity of a system like VA-TAMP is justified relative to simpler methods, such as TiPToP, as well as to ablations of VA-TAMP. Our ablations and tasks are designed to explore the following questions:

Q1: Do planned views derived from the current scene estimate generate useful targets for active perception?

Q2: Does a task-centric approach to model refinement improve downstream planning performance?



(a) Cubes on Bread (b) Teacup on Plate (c) Supplies on Tray

Fig. 4: The initial wrist-camera view of each task.

TABLE I: Success rate (successful trials / attempted trials) for each method on each scene.

Method	SR ↑		
	Cubes on Bread	Teacup on Plate	Supplies on Tray
TiPToP [4]	2 / 10	3 / 10	0 / 10
VUTAMP-UNIFORM	8 / 10	3 / 10	2 / 10
VUTAMP	8 / 10	9 / 10	2 / 10
VA-TAMP	9 / 10	10 / 10	10 / 10

We answer **Q1** by defining the visibility-unaware ablation VUTAMP, which does not actively plan observation actions or derive visibility preconditions for replanning. Instead, the memory provides a set of mesh estimates used for fully observed planning, and replans are only triggered if a memory update invalidates the current plan by introducing a collision (this happens when the memory update grows an obstacle, and a nearby collision-free configuration now collides with the grown obstacle).

We answer **Q2** by defining the VUTAMP-UNIFORM ablation. This ablation follows the same planning and replanning loop as VUTAMP, but the update strategy for the OBM is task-agnostic. Every object in the scene is considered equally relevant and the OBM focuses on refining whichever slot has the highest error with respect to the observation sequence.

Finally, we compare against TiPToP [4], an open-loop model-based planning system that composes foundation models to estimate the initial scene. TiPToP forms initial conservative estimates of the scene and executes the generated plan open-loop, without updating on new observations.

A. Experimental Setup

We perform evaluations on the Franka FR3 with a ZED-M wrist camera with known intrinsics and extrinsics. The perception pipeline uses Gemini-ER-2 [36] as the VLM, SAM2 [37] as the segmentation model, and RecGen [10] as the image-to-3D reconstruction model. The fully observed TAMP solver we use is cuTAMP [2], a GPU-parallelized TAMP solver. Our illustrative manipulation settings are shown in Figure 4.

In **Cubes on Bread**, the robot must pick up two stacked blocks and place both of them on the slice of bread. Moving the top block reveals the extent of the bottom block.

In **Teacup on Plate**, the robot has to move the blue cup from the table and to the flower-shaped plate. To do this, it must first take the occluding block off the cup.

The **Supplies on Tray** task features two cleaning supplies that are occluded by the boxes in the scene, which the robot has to look behind. The tray that the supplies have to be placed on also has collision obstacles that are not visible from the initial view.

B. Results

Our results are in Table I. We report the success rate of each method on each task. We find that each layer of added task complexity motivates another component of VA-TAMP. In the **Cubes on Bread** task, the bottom cube was occluded, which required a memory system to associate and update the initial bottom cube representation after revealing the full extent. TiPToP's failures on this task could be divided largely into grasp failures due to trying to place the robot's finger inside the body of the bottom cube, and placement failures due to trying to place the bottom cube onto a region of the toast that the top cube had already occupied. Because VUTAMP-UNIFORM was able to update its memories, it could outperform TiPToP.

In the **Cubes on Bread** task, there were only three objects in the scene and they were all task-relevant. The **Teacup on Plate** task introduces another layer of complexity by cluttering the scene with many task-irrelevant objects. An OBM that focused on greedily improving the worst mesh in the scene would miss the task-relevant objects, the teacup on the plate and the ones on the table the robot would pick up. By weighing the objects higher due to their proximity to the robot in its nominal plan, the objects are refined earlier by the OBM, leading to VUTAMP outperforming VUTAMP-UNIFORM.

In the **Supplies on Tray** task, the robot does not incidentally receive useful information while performing goal-oriented tasks. Unlike the preceding two tasks, where removing occluders was geometrically necessary to reach target objects, the cardboard boxes occlude the cleaning supplies visually but do not prevent the robot from reaching for those supplies. The two main failure modes for this task were grabbing an object about its longer axis (which seemed collision-free due to the cardboard box occlusion), or trying to place the objects on the red tray at an orientation that caused collision with the obstacles already there. In both cases, the robot had to plan observations which did not directly interact with the objects in the scene, but did serve to validate the robot's scene models. VA-TAMP is the only method that does this and thus has the strongest performance on this task.

VII. CONCLUSION

We presented VA-TAMP, an approach to *partially observable object rearrangement* that combines a persistent object-based memory with a deterministic TAMP solver augmented with information-gathering actions. Our tabletop experiments showed that both components were necessary in scenes with distractor objects, occlusion, and out-of-frame obstacles.

Our approach assumes that the initial observation supports a nominal plan that can guide information gathering. Integrating object search [38], [39] could extend it to settings in which some objects are missing from the scene model.

We currently use generative shape models conditioned on a single image. Models that incorporate multiple observations [40], [41] offer a promising alternative, but their inference time scales poorly with the number of views. Selecting informative or diverse subsets of observations could help make these models practical within the OBM. Finally, reconstruction methods that combine RGBD observations with contact [42], [43], [44] could extend plan justification to include touch-based verification.

ACKNOWLEDGMENTS

Under the authors' direction, OpenAI Codex helped copy-edit Sections I–V and the TikZ code for Figure 2. Anthropic Claude assisted with logging code for experiment assets. The authors take full responsibility for the paper.

REFERENCES

[1] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, "PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning," in *ICAPS*, 2020.

[2] W. Shen, C. Garrett, N. Kumar, A. Goyal, T. Hermans, L. P. Kaelbling, T. Lozano-Pérez, and F. Ramos, "Differentiable gpu-parallelized task and motion planning," *arXiv:2411.11833*, 2024.

[3] A. Curtis, X. Fang, L. P. Kaelbling, T. Lozano-Pérez, and C. R. Garrett, "Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances," in *ICRA*, 2022.

[4] W. Shen, N. Kumar, S. Chintalapudi, R. Lindeborg, J. Wang, C. Watson, E. S. Hu, J. Cao, D. Jayaraman, L. P. Kaelbling, and T. Lozano-Pérez, "TiPToP: A modular open-vocabulary robot manipulation system that plans," *arXiv:2603.09971*, 2026.

[5] A. Curtis, L. Kaelbling, and S. Jain, "Task-directed exploration in continuous pomdps for robotic manipulation of articulated objects," *arXiv:2212.04554*, 2022.

[6] A. Bry and N. Roy, "Rapidly-exploring random belief trees for motion planning under uncertainty," in *ICRA*, 2011.

[7] R. Platt, L. Kaelbling, T. Lozano-Perez, and R. Tedrake, "Efficient planning in non-gaussian belief spaces and its application to robot grasping," in *ISRR*, 2016.

[8] K. Hauser, "Randomized belief-space replanning in partially-observable continuous spaces," in *WAFR*, 2010.

[9] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, *et al.*, "Sam 3d: 3dfy anything in images," in *CVPR*, 2026.

[10] A. Zadaianchuk, L. Barcellona, L. Schuenemann, C. Gumbsch, Z. Wang, M. Z. Irshad, F. Despinoy, R. Aljundi, S. Gavves, and S. Zakharov, "Reconstruction by generation: 3d multi-object scene reconstruction from sparse observations," arXiv:2604.27106, 2026.

[11] G. Goretkin, L. P. Kaelbling, and T. Lozano-Pérez, "Look before you sweep: Visibility-aware motion planning," in *WAFR*, 2018.

[12] J. Muguira-Iturralde, A. Curtis, Y. Du, L. P. Kaelbling, and T. Lozano-Pérez, "Visibility-aware navigation among movable obstacles," in *ICRA*, 2023.

[13] S. Ashur and A. Sintov, "Sampling-based visibility task planning," *IEEE RA-L*, 2026.

[14] M. Lauri, J. Pajarinen, J. Peters, and S. Frintrop, "Multi-sensor next-best-view planning as matroid-constrained submodular maximization," *IEEE RA-L*, 2020.

[15] J. Delmerico, S. Isler, R. Sabzevari, and D. Scaramuzza, "A comparison of volumetric information gain metrics for active 3d object reconstruction," *Auton. Robots*, 2018.

[16] D. Hadfield-Menell, E. Groshev, R. Chitnis, and P. Abbeel, "Modular task and motion planning in belief space," in *IROS*, 2015.

[17] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox, "Online replanning in belief space for partially observable task and motion problems," in *ICRA*, 2020.

[18] L. P. Kaelbling and T. Lozano-Pérez, "Integrated task and motion planning in belief space," *IJRR*, 2013.

[19] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Partially observable task and motion planning with uncertainty and risk awareness," in *RSS*, 2024.

[20] L. Zhao, W. McClinton, A. Curtis, N. Kumar, T. Silver, L. P. Kaelbling, and L. L. Wong, "Seeing is believing: Belief-space planning with foundation models as uncertainty estimators," *arXiv:2504.03245*, 2025.

[21] T. Tanaka, M. Stephenson, A. Macvicar, and E. Simo-Serra, "Evidence-gated task and motion planning with vision-language models," *arXiv:2608.20084*, 2026.

[22] A. Oloo, Z. Altaweel, Y. Hayamizu, P. Liu, Y. Ding, S. Amiri, H. Yang, A. Kaminski, C. Esselink, C. Paxton, X. Zhang, and S. Zhang, "Robot planning and situation handling with active perception," in *IROS*, 2026.

[23] C. Mitash, R. Shome, B. Wen, A. Boularias, and K. Bekris, "Task-driven perception and manipulation for constrained placement of unknown objects," *IEEE RA-L*, 2020.

[24] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, *et al.*, "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," in *ICRA*, 2024.

[25] Y. Du, T. Lozano-Perez, and L. P. Kaelbling, "Learning object-based state estimators for household robots," in *IROS*, 2022.

[26] A. Agarwal, G. Singh, B. Sen, T. Lozano-Pérez, and L. P. Kaelbling, "Scenecomplete: Open-world 3d scene completion in cluttered real world environments for robot manipulation," *IEEE RA-L*, 2026.

[27] W.-C. Huang, J. Han, X. Ye, Z. Pan, and K. Hauser, "Simulation-ready cluttered scene estimation via physics-aware joint shape and pose optimization," *arXiv:2602.20150*, 2026.

[28] X. Yu, R. Talak, L. Shaikewitz, and L. Carlone, "Picasso: Holistic scene reconstruction with physics-constrained sampling," *arXiv:2602.08058*, 2026.

[29] H. Moravec and A. Elfes, "High resolution maps from wide angle sonar," in *ICRA*, 1985.

[30] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, 1989.

[31] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, "Dinov2: Learning robust visual features without supervision," *arXiv:2304.07193*, 2023.

[32] S. Ren, J. Hou, X. Chen, H. Xiong, and W. Wang, "Ddm: A metric for comparing 3d shapes using directional distance fields," *IEEE TPAMI*, 2025.

[33] N. Gothoskar, M. Ghavami, E. Li, A. Curtis, M. Noseworthy, K. Chung, B. Patton, W. T. Freeman, J. B. Tenenbaum, M. Klukas, *et al.*, "Bayes3d: fast learning and inference in structured generative models of 3d objects and scenes," *arXiv:2312.08715*, 2023.

[34] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Auton. Robots*, 2013.

[35] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio, and K. Goldberg, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," in *RSS*, 2017.

[36] Gemini Robotics Team, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, T. Armstrong, A. Balakrishna, R. Baruch, M. Bauza, M. Blokzijl, *et al.*, "Gemini robotics: Bringing ai into the physical world," *arXiv:2503.20020*, 2025.

[37] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, *et al.*, "Sam 2: Segment anything in images and videos," in *ICLR*, 2025.

[38] K. Zheng, A. Paul, and S. Tellex, "A system for generalized 3d multi-object search," in *ICRA*, 2023.

[39] D. Honerkamp, M. Büchner, F. Despinoy, T. Welschehold, and A. Valada, "Language-grounded dynamic scene graphs for interactive object search with mobile manipulation," *IEEE RA-L*, 2024.

[40] B. Li, D. Wu, J. Li, S. Zhou, Z. Zeng, L. Li, and H. Zha, "Mv-sam3d: Adaptive multi-view fusion for layout-aware 3d generation," *arXiv:2603.11633*, 2026.

[41] Y. Zhang, Y. Luo, X. Sun, F. Zhang, Z. Huang, and X. Tan, "Scene-sam3d: Multi-view scene asset generation without fine-tuning," *arXiv:2607.16805*, 2026.

[42] B. Bianchini, M. Zhu, M. Sun, B. Jiang, C. J. Taylor, and M. Posa, "Vysics: Object reconstruction under occlusion by fusing vision and contact-rich physics," *arXiv:2504.18719*, 2025.

[43] B. Saund and D. Berenson, "Clasp: Constrained latent shape projection for refining object shape from robot contact," in *CoRL*, 2022.

[44] E. Smith, D. Meger, L. Pineda, R. Calandra, J. Malik, A. Romero Soriano, and M. Drozdzal, "Active 3d shape reconstruction from vision and touch," *NeurIPS*, 2021.